

DATA ANALYSIS INTERNSHIP

DAY 12 REPORT

Matplotlib – Range Functions, Graph Types & Comparisons

Introduction

Matplotlib is a Python library used for data visualization. It helps to represent numerical data in graphical form for analysis and understanding. It supports different types of graphs and axis control functions which help in customizing plots.

Import Statement

```
import matplotlib.pyplot as plt
```

```
import random
```

```
import numpy as np
```

- matplotlib.pyplot → Used for graph plotting
 - random → Used to generate random values
 - numpy → Used for mathematical operations
-

Task 1 – Range Functions in Matplotlib

1. Using plt.xlim() and plt.ylim()

```
plt.plot(x,y)
```

```
plt.xlim(0,10)
```

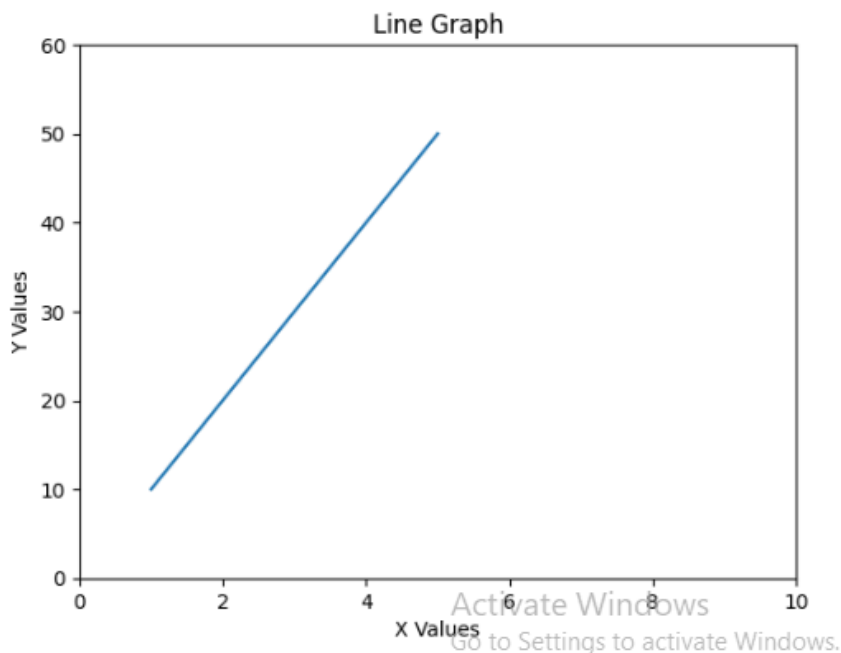
```
plt.ylim(0,60)
```

Purpose:

- Controls X-axis and Y-axis range separately
- Used for zoom in / zoom out effect

X Values : [1, 2, 3, 4, 5]

Y Values : [10, 20, 30, 40, 50]



2. Using plt.axis()

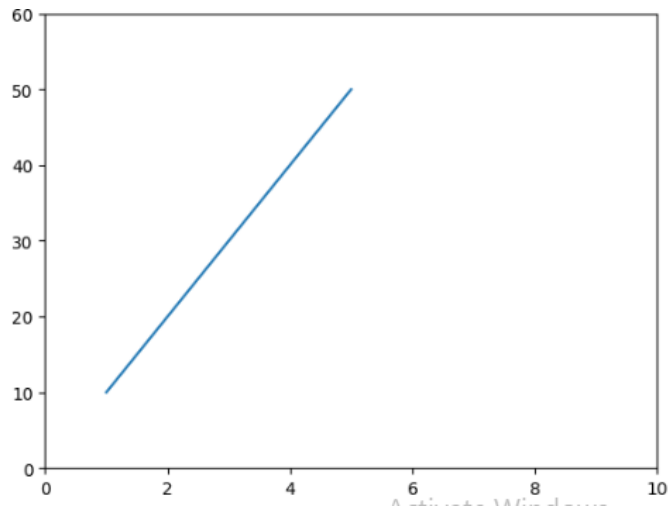
```
plt.plot(x,y)
```

```
plt.axis([0,10,0,60])
```

Purpose:

- Sets both X and Y axis range in one line

- Format: [xmin, xmax, ymin, ymax]



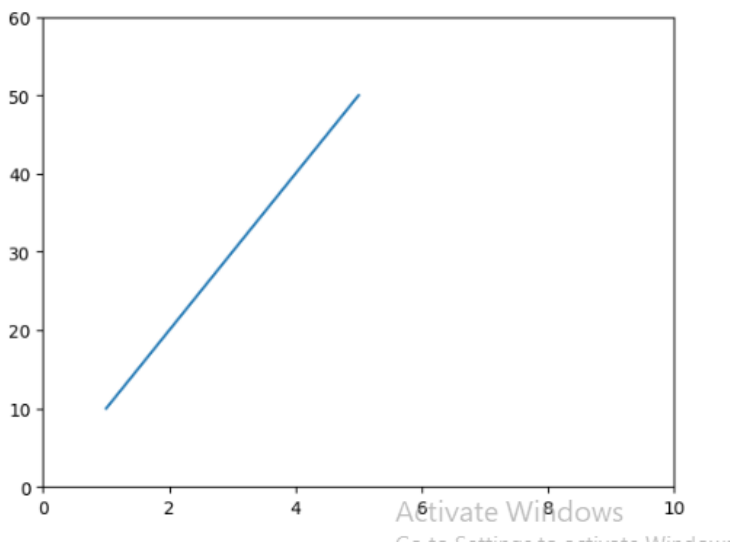
3. Using plt.xticks() and plt.yticks()

`plt.xticks([0,2,4,6,8,10])`

`plt.yticks([0,10,20,30,40,50,60])`

Purpose:

- Controls tick values shown on axis
- Helps in customizing scale display



Task 2 – Subplot (2 Graphs in One Figure)

```
plt.subplot(1,2,1)
```

```
plt.plot(x,y)
```

```
plt.subplot(1,2,2)
```

```
plt.bar(a,b)
```

Purpose:

- Displays multiple graphs in one window
- Format: rows, columns, position

Task 3 – Sin and Cos Multiplication Graph

```
x = random.randint(1,50)
```

```
y = random.randint(1,50)
```

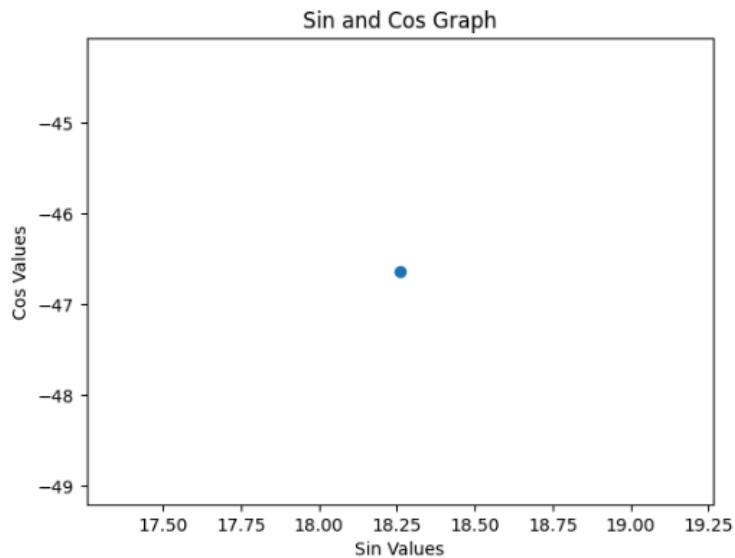
```
a = x * np.sin(x)
```

```
b = y * np.cos(y)
```

```
plt.plot(a,b,"o")
```

Purpose:

- Generates mathematical pattern using sin and cos
- Used for scientific visualization



Task 4 – Different Graph Types

1. Stem plot:

```
import matplotlib.pyplot as plt
```

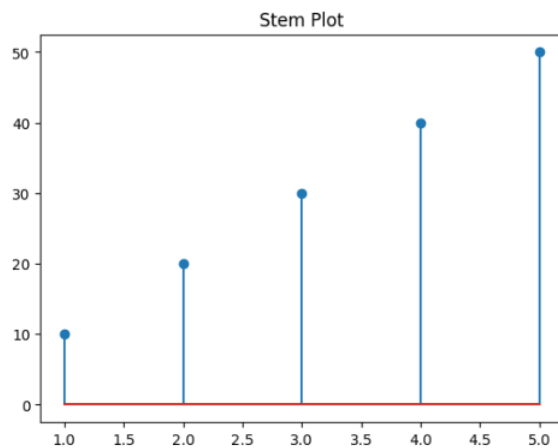
```
x = [1,2,3,4,5,6,7]
```

```
y = [10,25,15,40,30,50,45]
```

```
plt.stem(x,y)
```

```
plt.title("Stem Plot")
```

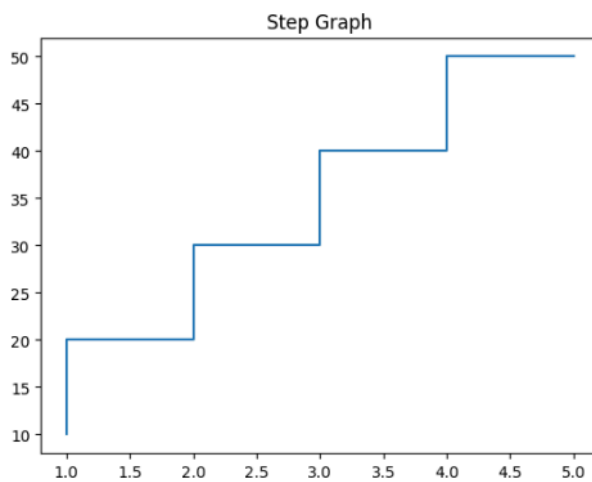
```
plt.show()
```



2. Step Graph

`plt.step(x,y)`

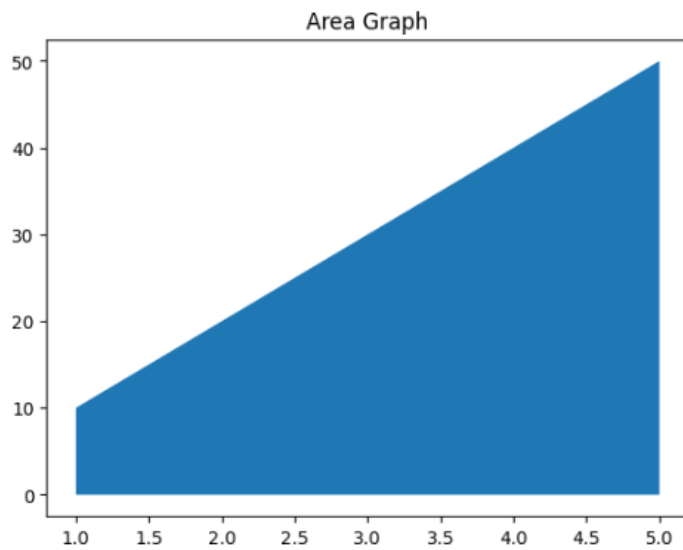
- Displays step-wise data changes



3. Area Graph

`plt.fill_between(x,y)`

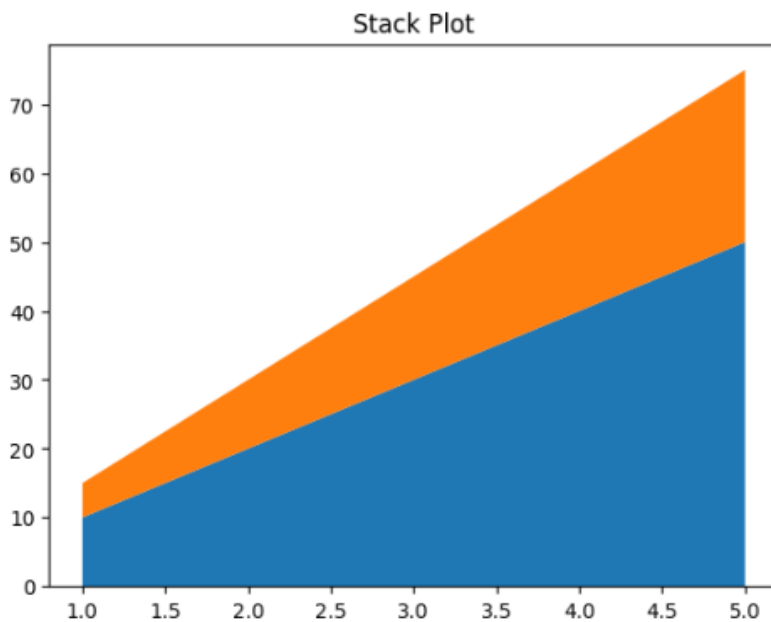
- Shows filled area under curve



4. Stack Plot

`plt.stackplot(x,y1,y2)`

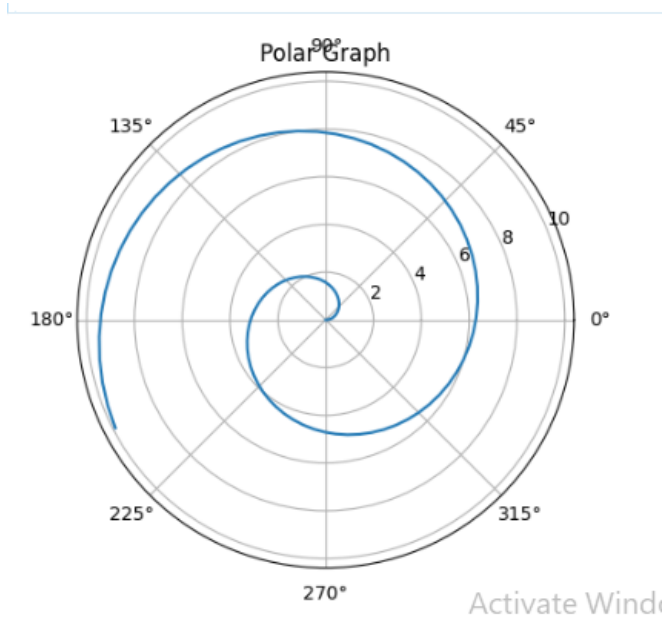
- Shows multiple datasets stacked together



5. Polar Graph

`plt.polar(x,y)`

- Used for circular / angular data visualization



6. Pie Chart

```
import matplotlib.pyplot as plt
values = [20,30,25,25]
labels = ["A","B","C","D"]
plt.pie(values, labels=labels)
plt.title("Pie Chart")
plt.show()
```

Pie Chart



Task 5 – Comparison of X vs Y and X vs Z

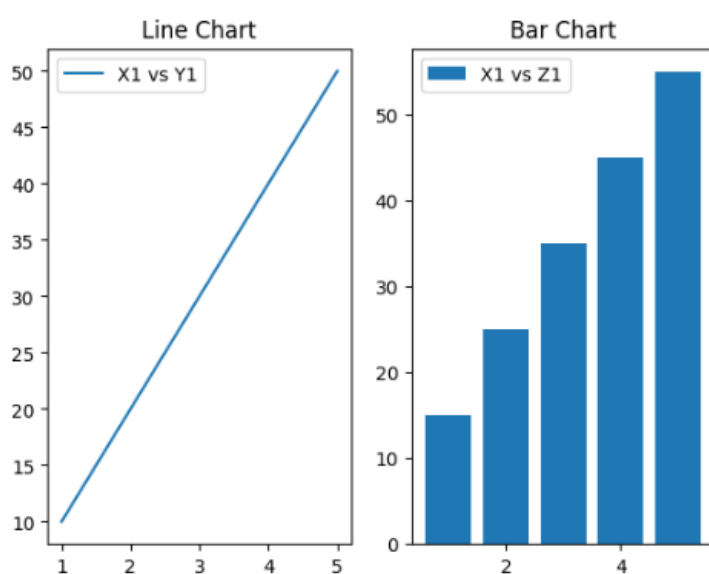
```
import matplotlib.pyplot as plt
x1 = [1,2,3,4,5]
y1 = [10,20,30,40,50]
z1 = [15,25,35,45,55]
plt.subplot(1,2,1)
plt.plot(x1,y1,label="X1 vs Y1")
plt.title("Line Chart")
plt.legend()
plt.subplot(1,2,2)
plt.bar(x1,z1,label="X1 vs Z1")
plt.title("Bar Chart")
```

`plt.legend()`

`plt.show()`

Purpose:

- Compares two datasets
- One using line graph and another using bar graph
- Helps in visual comparison of values



Conclusion

In this session, different graph plotting techniques were practiced using Matplotlib. Various range control functions like `xlim`, `ylim`, `axis`, and `tick` functions were learned.

Multiple graph types such as line, bar, histogram, step, area, stack, and polar plots were implemented. Subplot and comparison graphs helped in understanding multi-graph visualization techniques.

